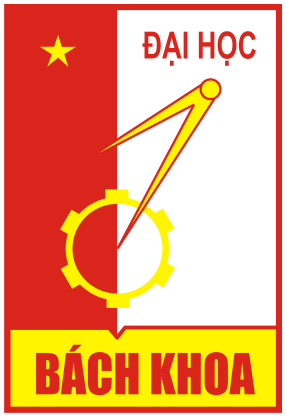
Hanoi University of Science and Technology

School of Informations and Communication Technology



Research Topic 16 - Harvest Planning

Maximizing harvested products

Group 10:

Nguyễn Duy Phúc – 20235616
Trần Quang Trọng – 20235565
Nguyễn Xuân Khải - 20235595
Ngô Minh Quang – 20235550

Table of Contents

1.Problem description

HARVEST PLANNING

Maximizing harvested products

Given N fields 1, 2, 3, …, N growing the same type of agricultural products.

The i^{th} field has a yield of d(i) and needs to be harvested on one day from day s(i) to day e(i).

Given these constraints:

- The processing factory has capacity M, which is the maximum number of harvested products it can process each day.
- If the total amount of products available is less than m, the factory will not work (as the efficiency is not good).
- A field can be selected to be harvested or not

Compute a plan for harvesting these fields so that:

- The total number of products harvested is maximum

A solution is represented by x[1], x[2], x[3], … x[N] in which x[i] is the day the field i is harvested. Leave x[i] = -1 if the field is not harvested.

2. Modeling

2.1. Exact Algorithms

2.1.1. CP model

Overview: We use OR-Tools CP-SAT model to find the optimal solution

Decis(i)on variables:

- x[i, j] = 1: Field i is harvested on day j
- x[i, j] = 0: Field i is not harvested on day j

```
x = {}
for i in range(N):
    for j in range(MD):
        x[i, j] = model.NewBoolVar(f'x_{i}_{j}')
```

- harvested_today[j] = 1: At day j, at least 1 field is harvested
- harvested_today[j] = 0: At day j, no field is harvested

```
harvest_today = {}
for j in range(MD):
    harvest_today[j] = model.NewBoolVar(f'harvest_today_{j}')
```

- m: The minimum product for the factory work
- M: The maximum product factory can process each day
- s_i, e_i: The field i can be harvested from day s_i to e_i
- MD: The max day field can be harvested

Constraints:

- A field can be harvested only once

$$\sum_{j=1}^{MD} x[i,j] \leq 1$$

```
for i in range(N):
    model.Add(sum(x[i, j] for j in range(MD)) <= 1)
```

- The harvested day of field i must fall within the range [s, e]
- The total amount of each day must be satisfied in range [m, M]

$$m \leq \sum_{i=1}^{N} (d[i] \times x[i,j]) \leq M$$

```
for j in range(MD):
    # Nếu có ít nhất một thửa được thu hoạch vào ngày j
    model.Add(sum(x[i, j] for i in range(N)) >= 1).OnlyEnforceIf(harvest_today[j])
    # Nếu không có thửa nào được thu hoạch vào ngày j
    model.Add(sum(x[i, j] for i in range(N)) == 0).OnlyEnforceIf(harvest_today[j].Not())
    amount = sum(harvest_plans[i][0] * x[i, j] for i in range(N))
    model.Add(amount >= m).OnlyEnforceIf(harvest_today[j])
    model.Add(amount <= M).OnlyEnforceIf(harvest_today[j])
```

Objective function:

- Maximize the total amount of product

$$\text{Objective} = \text{Max} \sum_{j=1}^{MD} \sum_{i=1}^{N} d[i] \times x[i,j]$$

```
# Objective function
model.Maximize(sum(harvest_plans[i][0] * x[i, j] for i in range(N) for j in range(MD)))
```

2.1.2 IP Model

Overview: We use OR-Tools SCIP model to find the optimal solution

Decision variables:

- x[i, j] = 1: Field i is harvested on day j
- x[i, j] = 0: Field i is not harvested on day j

```
x = {}
for i in range(N):
    for j in range(fields[i][1], fields[i][2] + 1):
        x[i, j] = solver.BoolVar(f'x[{i},{j}]')
```

- y[j] = 1: Factory works on day j
- y[j] = 0: Factory doesn't work on day j

```
for j in range(1, MD + 1):
    y[j] = solver.BoolVar(f'y[{j}]')
```

- m: The minimum of product for the factory work
- M: The maximum product factory can process each day
- fields[i][1], fields[i][2]: The field i can be harvested from day fields[i][1] to fields[i][2]
- MD: the last day the field can be harvested

Constraints:

- A field can be harvested only once

$$\sum_{j=1}^{MD} x[i,j] \leq 1$$

```
for i in range(N):
    solver.Add(sum(x[i, j] for j in range(fields[i][1], fields[i][2] + 1)) <= 1)
```

- If the factory works on day j, at least 1 field is harvested

$$\sum_{i=1}^{N} x[i,j] \leq y[j] \times N \qquad \sum_{i=1}^{N} x[i,j] \geq y[j]$$

```
for j in range(1, MD + 1):
    solver.Add(sum(x[i, j] for i in range(N) if fields[i][1] <= j <= fields[i][2]) <= y[j] * N)
for j in range(1, MD + 1):
    solver.Add(sum(x[i, j] for i in range(N) if fields[i][1] <= j <= fields[i][2]) >= y[j])
```

- The total amount of each day must be satisfied in range [m, M]

$$m \leq \sum_{i=1}^{N} (d[i] \times x[i,j]) \leq M$$

```
for j in range(1, MD + 1):
    solver.Add(sum(fields[i][0] * x[i, j] for i in range(N) if fields[i][1] <= j <= fields[i][2]) <= M * y[j])
total_harvested = sum(fields[i][0] * x[i, j] for i in range(N) for j in range(fields[i][1], fields[i][2] + 1))
solver.Add(total_harvested >= m)
```

Objective function:

- Maximize the total amount of product

$$\text{Objective} = \text{Max} \sum_{j=1}^{MD} \sum_{i=1}^{N} d[i] \times x[i,j]$$

```
total_harvested = sum(fields[i][0] * x[i, j] for i in range(N) for j in range(fields[i][1], fields[i][2] + 1))
solver.Maximize(total_harvested)
```

2.2. Greedy Algorithm

2.2.1. Objective

This code processes N fields, each with a daily yield di and a valid harvesting window from day s(i) through day e(i). It aims to decide on which day each field is harvested under two constraints:

- Daily minimum yield m required to initiate harvesting.
- Daily maximum yield M that cannot be exceeded.

2.2.2. Data Structures and Initialization

- **Input Handling**: Reads N, m, M. Then for each field i, it stores:
  - quantity_fields[i] = (d_i, e_i).
  - Adds i to s_days[s_i] and e_days[e_i + 1], marking when it becomes available and expires.

- **Arrays**:
  - s_days and e_days are lists of lists to track fields entering and leaving availability on each day.
  - day_harvest: an array of length N indicating the harvest day for each field (initially −1 if not harvested).

2.2.3. Main Loop (Day-by-Day Process(i)ng)

- For each day 1 to max_day:
  1. **Add Available Fields**: Include fields whose start day is today in cur_fields, sum up their yields.
  2. **Remove Expired Fields**: Exclude fields whose valid window ended before today if they are still not harvested.
  3. **Adjust Exceeding Yield**: If total yield exceeds M, move the fields with the largest ei (longest remaining window) to the next day's list (s_days[i+1]) until the total is within M.

```
        cur_fields.pop(field)
while cur_quantity > M:
    field = max(cur_fields, key=lambda x: cur_fields[x][1])
    if quantity_fields[field][1] == i:
        break
    cur_quantity -= cur_fields[field][0]
    s_days[i + 1].append(field)
    cur_fields.pop(field)
```

  4. **Harvest Decis(i)on**: If the remaining total yield ≥m , harvest all fields in cur_fields. Mark their harvest day, reset cur_fields.

2.2.4. Output

- Prints N.
- For each field, prints its 1-based index and the day it was harvested (or −1 if never harvested).

2.3. Greedy with Local Search

2.3.1. Importing defaultdict

The defaultdict from the collection's module is a type of dictionary in that initializes default values automatically if a key does not exist. In this code, it is used to store the daily harvest amounts (daily_harvest).

```
1   from collections import defaultdict
```

2.3.2. The Function harvest_plan_local_search

This function ass(i)gns fields to harvest days while satisfying the constraints, us(i)ng a **greedy initialization** followed by a **local search optimization**.

```
3   #vào code
4   def harvest_plan_local_search(N, m, M, fields):
```

Step 1: Initial Ass(i)gnment with a Greedy Approach

- **Objective**: Ass(i)gn each field to the earliest poss(i)ble day within its allowable range, ensuring the daily harvest does not exceed M (the maximum daily capacity).
- **Details**:
    - The fields are sorted based on this order:
        1. −x [1][0]: Harvest quantity (prioritize fields with higher yield).
        2. x [1][2] – x [1][1] Harvest period length (prioritize fields with shorter harvest windows).

```
5       # Step 1: khởi tạo greedy
6       fields = sorted(enumerate(fields, start=1), key=lambda x: (-x[1][0], x[1][2] - x[1][1]))
7       daily_harvest = defaultdict(int)
8       field_assignments = [-1] * (N + 1)
```

    - Each field is then ass(i)gned to the earliest day within its harvest window [s,e] that satisfies the constraint daily_harvest[t] + d ≤ M

```
10      for field_id, (d, s, e) in fields:
11          for day in range(s, e + 1):
12              if daily_harvest[day] + d <= M:
13                  daily_harvest[day] += d
14                  field_assignments[field_id] = day
15                  break
```

Step 2: Local Search Optimization

- **Function evaluate(day)**: Checks if the total harvest for a given day satisfies the minimum requirement m

```
17      # Step 2: Local search optimization
18      def evaluate(day):
19          return daily_harvest[day] >= m
```

- **Function reass(i)gn_field(field_id, d, s, e)**:

```
21      def reassign_field(field_id, d, s, e):
22          for day in range(s, e + 1):
23              if daily_harvest[day] + d <= M and evaluate(day):
24                  return day
25          return -1
```

  - The function defines how to explore neighboring solitions for a given field
    - This process ensures that only feas(i)ble neighboring solutions are cons(i)dered.
  - Tries to find a new day t∈[s,e] for a field that satisfies the constraints M and m.
  - Returns the new day if successful; otherwise, returns -1.

- **Optimization Process**:
  - Detail explanation of the code can be found on the comments section of the code here

```
27  for field_id, (d, s, e) in fields:
28      current_day = field_assignments[field_id]
29  if current_day == -1 or not evaluate(current_day):  # ktra xem field có cần reassign không
30      new_day = reassign_field(field_id, d, s, e)  # tìm 1 ngày tốt hơn để assign field
31      if new_day != -1 and new_day != current_day:  # nếu ngày mới tìm được valid và khác ngày cũ
32          if current_day != -1:  # xoá field khỏi ngày cũ nếu đã được assign
33              daily_harvest[current_day] -= d
34          daily_harvest[new_day] += d  # thêm field đó vào ngày mới
35          field_assignments[field_id] = new_day  # update nghiệm mới
```

  - Fields that are either unass(i)gned or ass(i)gned to a day that does not meet the minimum requirement are recons(i)dered.

- If a valid better day is found (one that satisfies M and m), the field is reass(i)gned to this day, and the harvest totals are updated.
- This adjustment explores and applies feas(i)ble neighboring solutions.

Step 3: Collecting Results

- The results cons(i)st of a list of tuples (field_id, day), where:
  - field_id: ID of the field.
  - day: The day the field is ass(i)gned, meeting all constraints

```
37    # Step 3: collect đáp án
38    result = [
39        (field_id, day)
40        for field_id, day in enumerate(field_assignments)
41        if day > 0 and evaluate(day)
42    ]
```

2.3.3. Input and Output

- **Input**:

```
50  # Input
51  N, m, M = map(int, input().split())
52  fields = [tuple(map(int, input().split())) for _ in range(N)]
53  harvest_plan_local_search(N, m, M, fields)
```

  - N, m, M: Number of fields, minimum daily harvest, and maximum daily harvest.
  - A list of fields, where each field is represented as (d,s,e) (yield, start day, end day).

- **Output**:

```
44    # Output
45    print(len(result))
46    for field_id, day in result:
47        print(field_id, day)
```

  - Prints the number of fields successfully ass(i)gned.
  - Prints a list of (field_id, day) pairs.

2.4. Simulated Annealing

Initial Heuristic Solution

This step constructs a **greedy initial solution** by prioritizing fields with higher did_idi and smaller harvest windows.

- Fields are sorted by:
  - Descending d(i) (fields with higher yields are cons(i)dered first).
  - Shorter e(i) – s(i) (fields with tighter time constraints are prioritized).

Assignment Rule: For each field i with d(i), s(i), e(i) :

1. Assign it to the earliest day t ∈ [s(i), e(i)] where:

$$\sum_{j:T_j=t} d_j + d_i \leq M$$

2. Update $T_i = t$ and daily_harvest[t] += d_i.

```python
# Step 1: Initial Heuristic Solution
def initial_solution():
    sorted_fields = sorted(enumerate(fields, start=1), key=lambda x: (-x[1][0], x[1][2] - x[1][1]))
    daily_harvest = defaultdict(int)
    assignments = [-1] * (N + 1)

    for field_id, (d, s, e) in sorted_fields:
        for day in range(s, e + 1):
            if daily_harvest[day] + d <= M:
                daily_harvest[day] += d
                assignments[field_id] = day
                break
    return assignments, daily_harvest
```

Evaluate Solution

The evaluation function calculates the total product processed on valid days t, where:

$$m \leq \text{daily\_harvest}[t] \leq M$$

Formula:

$$\text{total\_product} = \sum_{t:m \le \text{daily\_harvest}[t] \le M} \text{daily\_harvest}[t]$$

```python
# Step 2: Evaluate Solution
def evaluate(daily_harvest):
    valid_days = [day for day in daily_harvest if m <= daily_harvest[day] <= M]
    total_product = sum(daily_harvest[day] for day in valid_days)
    return total_product
```

Generating Neighbor Solution

A neighbor solution is generated by randomly modifying the assignment of one field:

1. Select a random field i.
2. Remove its contribution from its current assigned day t, if any:

$$daily\_harvest[t] \text{ -= } d_i$$

3. Reassign it to:
   - A random valid day $t' \in [s(i), e(i)]$ where $daily\_harvest[t'] + d(i) \le M$, *or*
   - -1 (not harvested).

This generates a **new solution** $(T', daily\_harvest')$.

```python
# Step 3: Generate Neighbor Solution
def neighbor_solution(assignments, daily_harvest):
    new_assignments = assignments[:]
    new_daily_harvest = daily_harvest.copy()

    # Randomly pick a field to modify
    field_id = random.randint(1, N)
    current_day = new_assignments[field_id]
    d, s, e = fields[field_id - 1]

    # Remove the field from its current day
    if current_day != -1:
        new_daily_harvest[current_day] -= d

    # Reassign to a new day or skip harvesting
    valid_days = [-1] + [day for day in range(s, e + 1) if new_daily_harvest[day] + d <= M]
    new_day = random.choice(valid_days)
    if new_day != -1:
        new_daily_harvest[new_day] += d
    new_assignments[field_id] = new_day

    return new_assignments, new_daily_harvest
```

Validate Solution

Before accepting a neighbor solution, ensure:

1. $daily\_harvest[t] \leq M \ \forall t$
2. $daily_{harvest[t]} \geq M \ \forall t \ where \ harvest \ occurs$

```python
# Step 4: Validate Solution
def is_valid_solution(daily_harvest):
    for day, harvest in daily_harvest.items():
        if harvest > M or (0 < harvest < m):
            return False
    return True
```

Simulated Annealing

We optimize the solution iteratively by exploring neighbors, accepting better solutions, and occas(i)onally accepting worse solutions to escape local optima.

Initialization:

- Start with the initial solution $(T, daily\_harvest)$

- Initial temperature $T_{\text{init}} = 100$
- Cooling rate $\alpha = 0.99$.
- Iterations $n_{\text{iter}} = 1000$.

Acceptance Criteria:

Let $\Delta = \text{new\_value} - \text{current\_value}$

- Accept the neighbor solution if $\Delta > 0$ (better solution).
- Otherwise, accept with probability: $P = e^{\Delta/T}$

Cooling Schedule:

Update temperature after each iteration:

$$T \leftarrow \alpha \cdot T$$

Steps:

1. Generate a neighbor solution $(T', \text{daily\_harvest}')$.
2. Evaluate new_value using the evaluation function.
3. Decide whether to accept $(T', \text{daily\_harvest}')$ based on the acceptance criteria.
4. Track the best solution found so far.

```python
# Step 5: Simulated Annealing
def simulated_annealing():
    assignments, daily_harvest = initial_solution()
    current_value = evaluate(daily_harvest)
    best_assignments = assignments[:]
    best_value = current_value

    temperature = 100
    cooling_rate = 0.99
    iterations = 1000

    for _ in range(iterations):
        new_assignments, new_daily_harvest = neighbor_solution(assignments, daily_harvest)
        if not is_valid_solution(new_daily_harvest):
            continue  # Skip invalid solutions

        new_value = evaluate(new_daily_harvest)
        delta = new_value - current_value

        # Accept new solution based on SA acceptance criteria
        if delta > 0 or random.uniform(0, 1) < (2.71828 ** (delta / temperature)):
            assignments, daily_harvest = new_assignments, new_daily_harvest
            current_value = new_value

            # Update best solution
            if current_value > best_value:
                best_assignments = assignments[:]
                best_value = current_value
```

Output Results

The best solution T_{best} is converted into:

- x[i] = t for each field i assigned to day t.
- x[i] = −1 for unharvested fields.

Print the total number of harvested fields and their assignments.

```python
# Step 6: Output the Best Solution
best_assignments, _ = simulated_annealing()
valid_assignments = [(field_id, day) for field_id, day in enumerate(best_assignments) if day != -1]

print(len(valid_assignments))
for field_id, day in sorted(valid_assignments):
    print(field_id, day)
```

3.Results and Analysis

3.1. Experimental Results:

Input size	Constraint Programming		Linear Programming		Greedy	
Number of fields	Amount of product	Runtime	Amount of product	Runtime	Amount of product	Runtime
n = 10	58	0.052295	58	0.062713	58	0.000044
n = 100	1020	0.147373	1020	0.084082	1020	0.000277
n = 1000	49589	2.271441	49589	0.485500	49589	0.015266
n = 5000	128868	36.31835	128868	2.54404	128868	0.015558
n = 10000	255386	85.13465	255386	5.651043	255386	0.083121

Input size	Greedy + Local Search		Simulated Annealing	
Number of fields	Amount of product	Runtime	Amount of product	Runtime
n = 10	58	0.000302	58	6.577999
n = 100	1020	0.000753	1020	3.327558
n = 1000	48450	0.005125	48450	26.331794
n = 5000	128573	0.023686	128573	26.09792
n = 10000	254638	0.037586	254638	32.921673

3.2. Analysis & Conclusion

For optimal product maximization (quality-focused):

- Use Constraint Programming or Linear Programming.
- Preferred for smaller to medium-sized inputs or when runtime is not a constraint.

For extremely fast solutions (time-focused):

- Use Greedy.
- Suitable for scenarios prioritizing speed over quality.

For a balance between quality and runtime:

- Use Greedy + Local Search.
- Recommended for large-scale problems where a good solution is needed quickly.

Simulated Annealing

- Can provide near-optimal solutions, but it requires significant runtime to achieve comparable results. Its exploratory nature makes it slower than Greedy and Greedy + Local Search while not necessarily surpassing them in quality.
- Runtime Efficiency: Simulated Annealing is computationally intensive and not suitable for scenarios requiring rapid solutions.

4.Member Contribution

4.1. Nguyễn Duy Phúc – 20235616

- Responsible for Exact Algorithms

- Responsible for Exact Algorithms slides and Exact Algorithms section in report

- Responsible for experimental result

4.2. Trần Quang Trọng – 20235565

- Responsible for Greedy Algorithm

- Responsible for Greedy Algorithm slides and Greedy Algorithm section in report

- Responsible for experimental result

4.3. Nguyễn Thiện Khải – 20235595

- Responsible for Greedy with Local Search Algorithm

- Responsible for Greedy with Local Search Algorithm slides and Greedy with Local Search Algorithm section in report

4.4. Ngô Minh Quang – 20235550

- Responsible for Simulated Annealing Algorithm

- Responsible for Simulated Annealing Algorithm slides and Simulated Annealing Algorithm section in report.

- Responsible for editing the Results and Analysis section and finalizing the report.